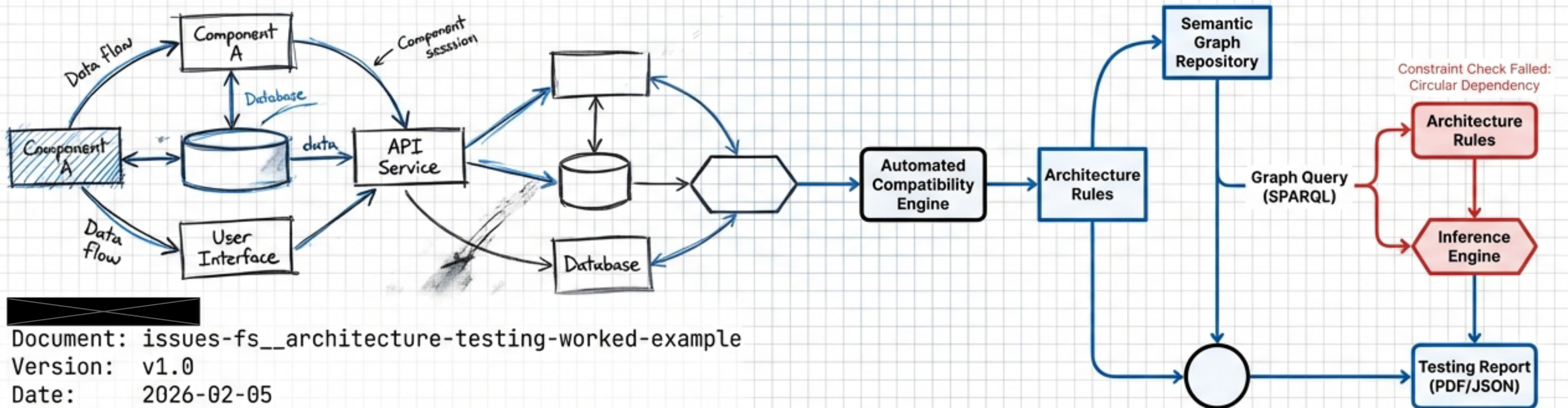


Architecture Testing with Semantic Graphs

A Complete Worked Example: From Diagrams to Automated Compatibility Reports.

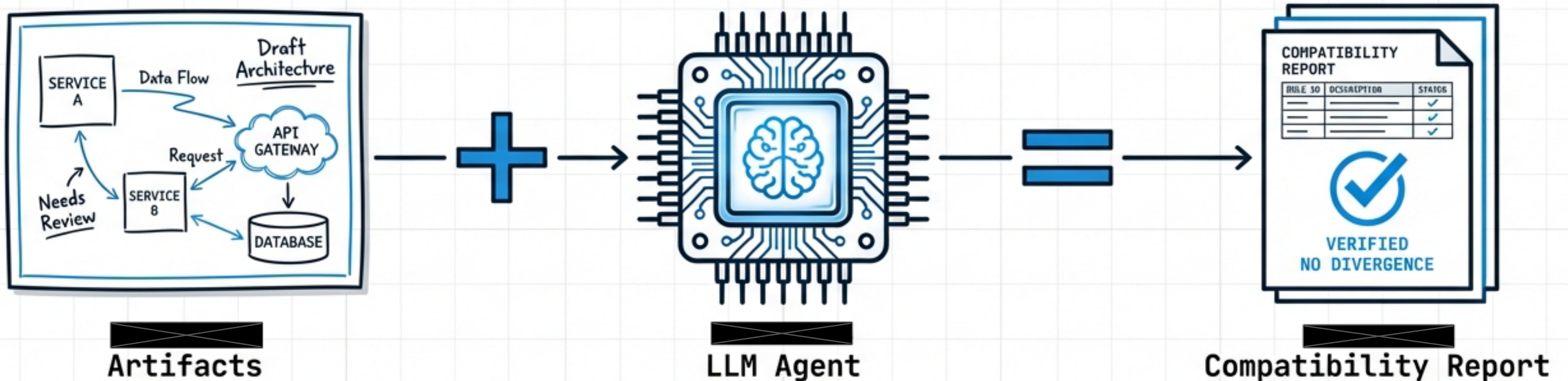


Document: issues-fs__architecture-testing-worked-example
Version: v1.0
Date: 2026-02-05
Status: Practical Guide

Turning "Boxes and Arrows" into Testable Truth

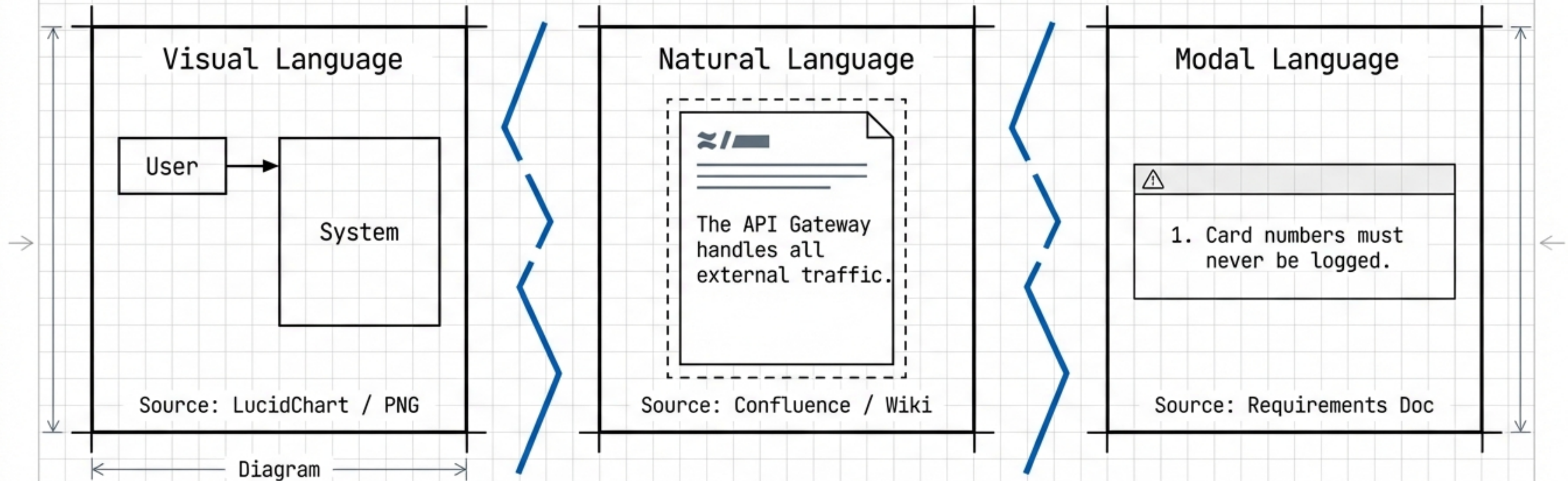
We demonstrate a live workflow where an **architect's natural artifacts**—**diagrams, text, and rules**—are **transformed into semantic graphs to automate consistency checking**.

- **No new languages:** Architects continue to use Context Diagrams, flow charts, and prose.
- **LLM as Execution Engine:** Agents act as the translation layer between human intent and machine verification.
- **Automated Verification:** Divergence between design and documentation is surfaced immediately.



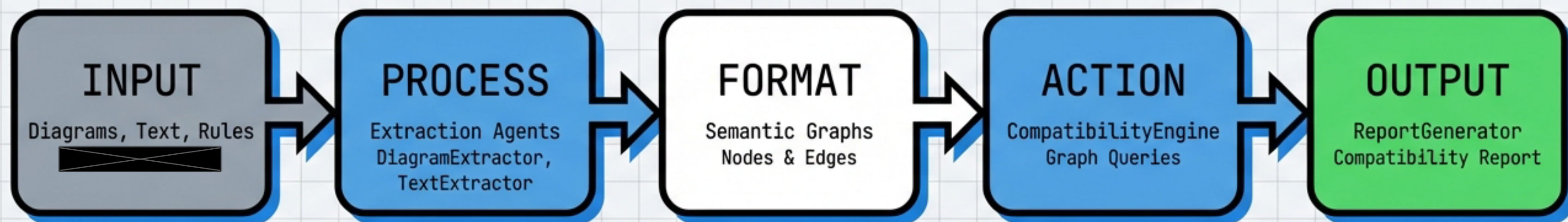
The Disconnect Between Intent and Artifact

Architects describe systems in three distinct “languages” that rarely agree.



The Friction: These artifacts live in different tools and drift apart over time. How do we automate the comparison?

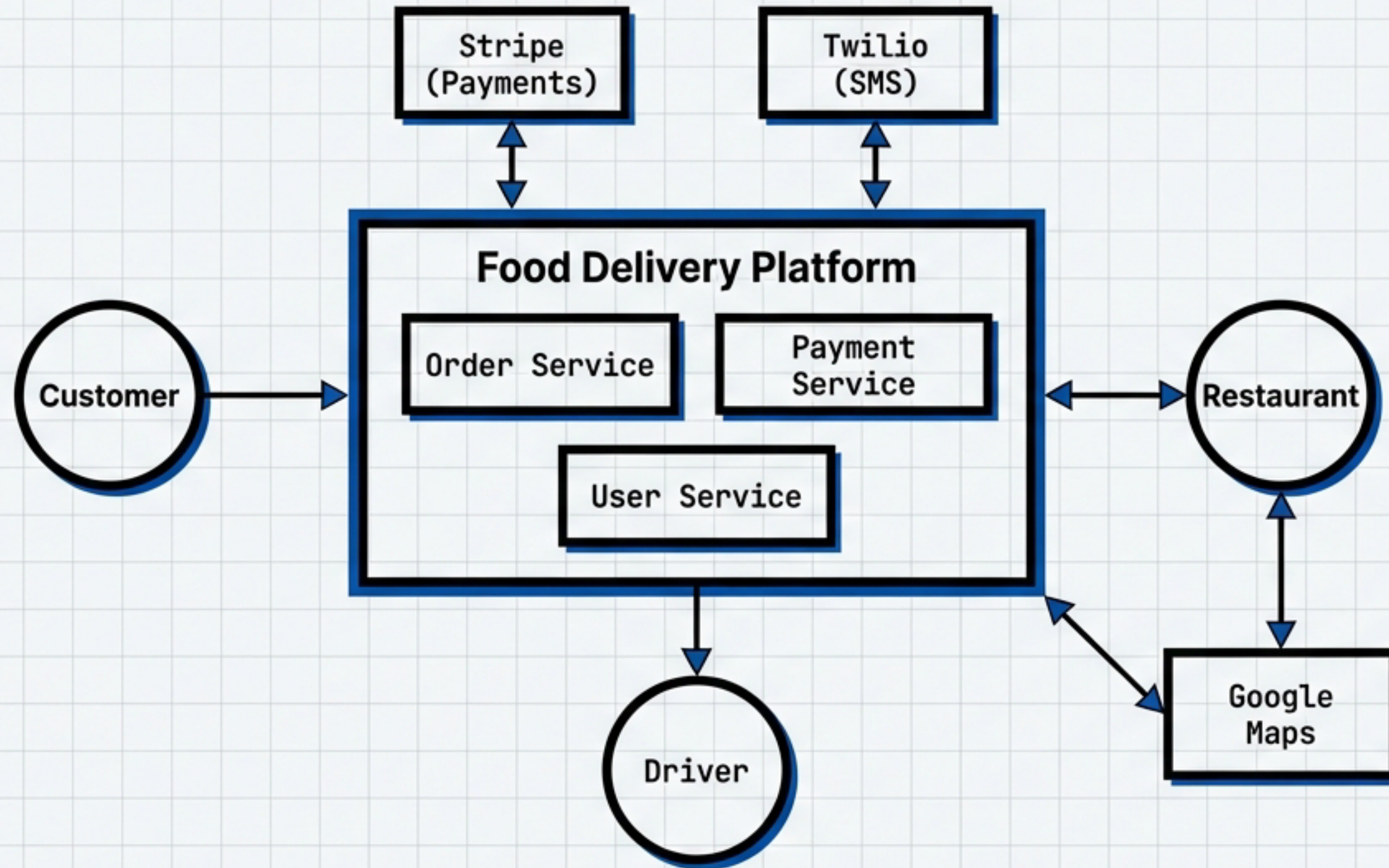
The Semantic Graph Pipeline



// We treat the LLM not as a creative writer, but as a deterministic execution engine for extraction and logic.

The Subject: A Modern Food Delivery Platform

A realistic system complex enough to break, but simple enough to follow.

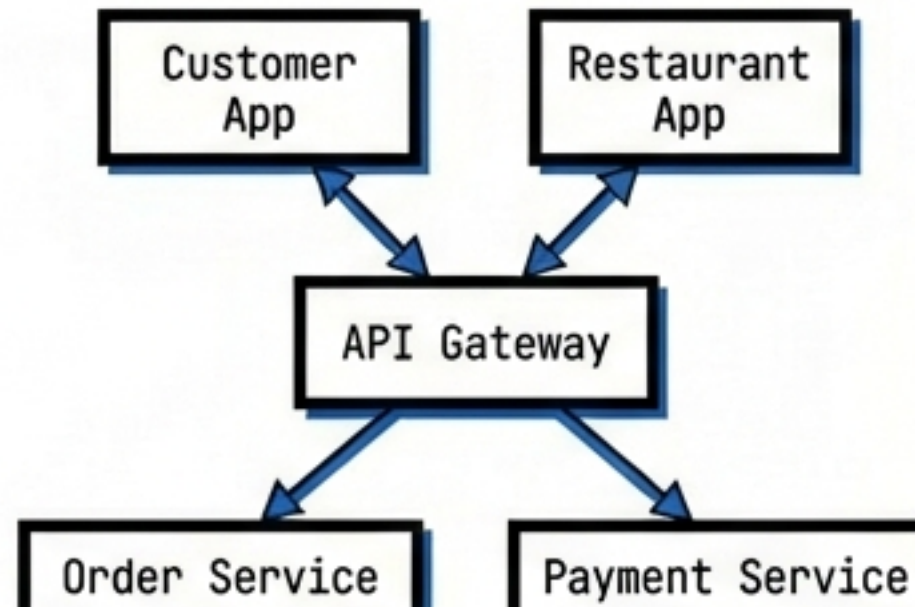


System Profile

- **Actors:** 3 (Customer, Restaurant, Driver)
- **Services:** 4 (User, Order, Payment, Notification)
- **Integrations:** Stripe, Twilio, Google Maps
- **Constraints:** PII Security, Order State Integrity

Inputs: What the Architect Actually Creates

Artifact 1: System Context Diagram.png



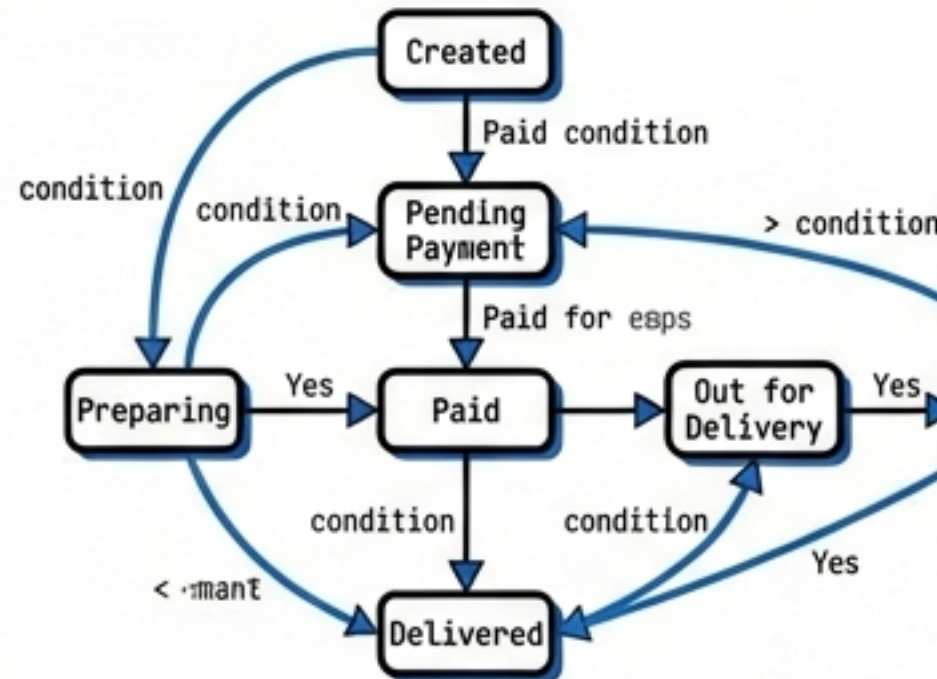
Artifact 7: Architecture_Rules.txt

MUST: External traffic -> API Gateway

Artifact 3: API_Gateway_Spec.md

The API Gateway acts as the single entry point for all external traffic. It handles authentication, rate limiting, and routing to internal microservices.

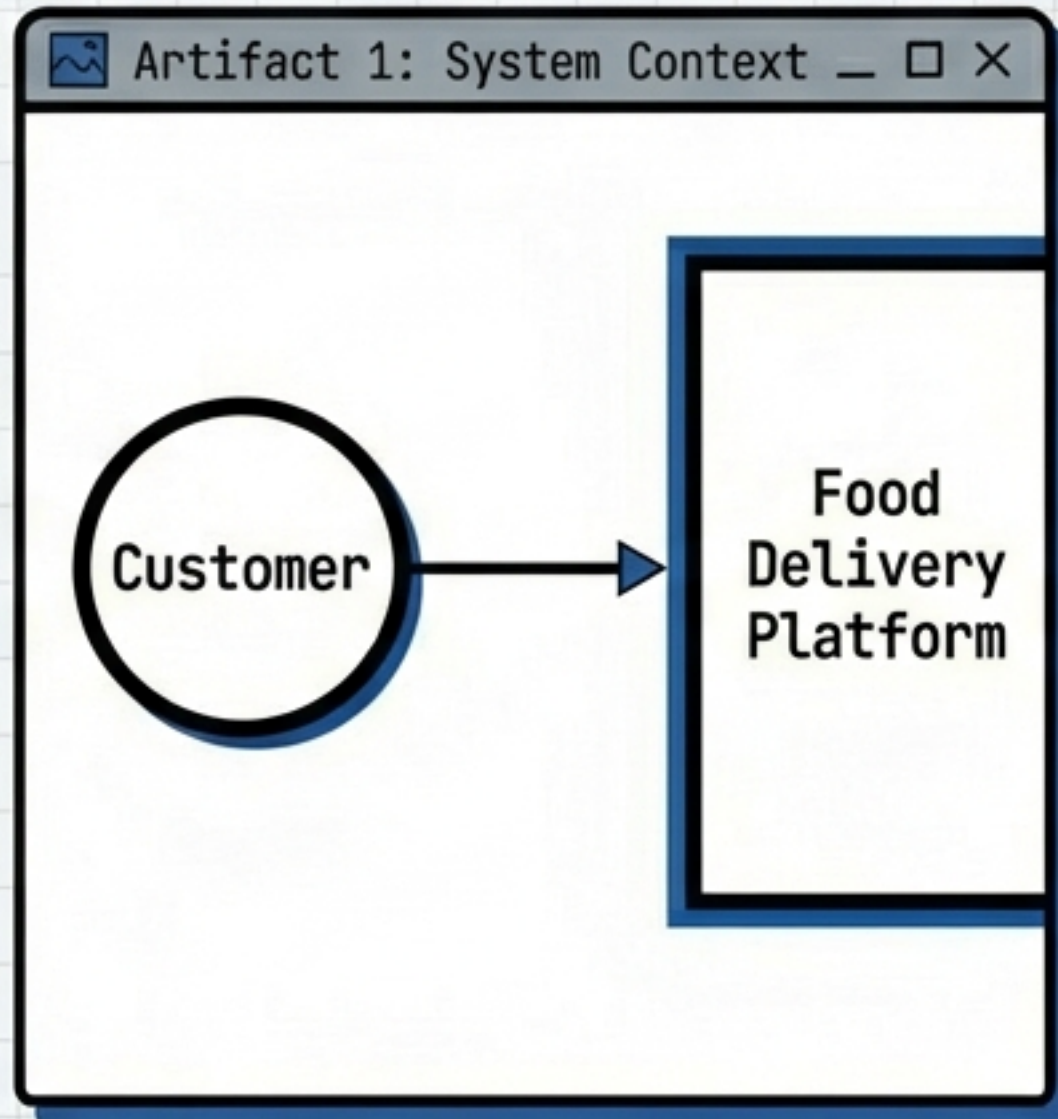
Artifact 5: Order_State_Machine.jpg



These are standard PNGs and Markdown files.
No metadata tags. No hidden code.

Step 1: Transforming Pixels into Nodes

Input



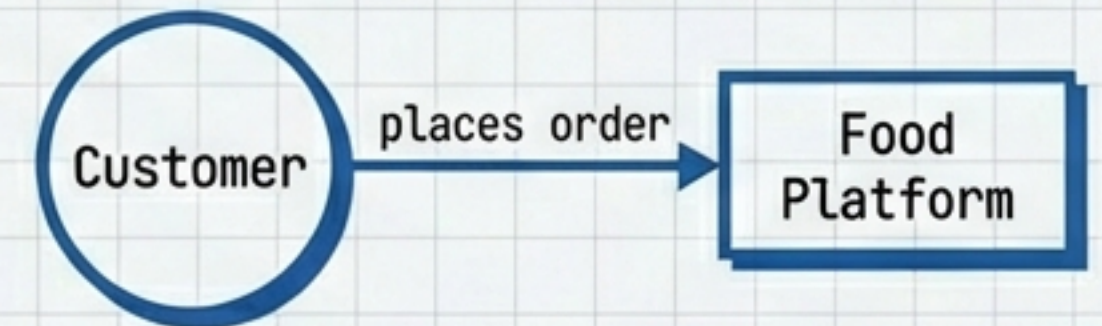
Artifact 1 (PNG)

The Prompt

> **System:** Act as a DiagramExtractor.
> **User:** Identify all actors and relationships in this image. Return JSON.

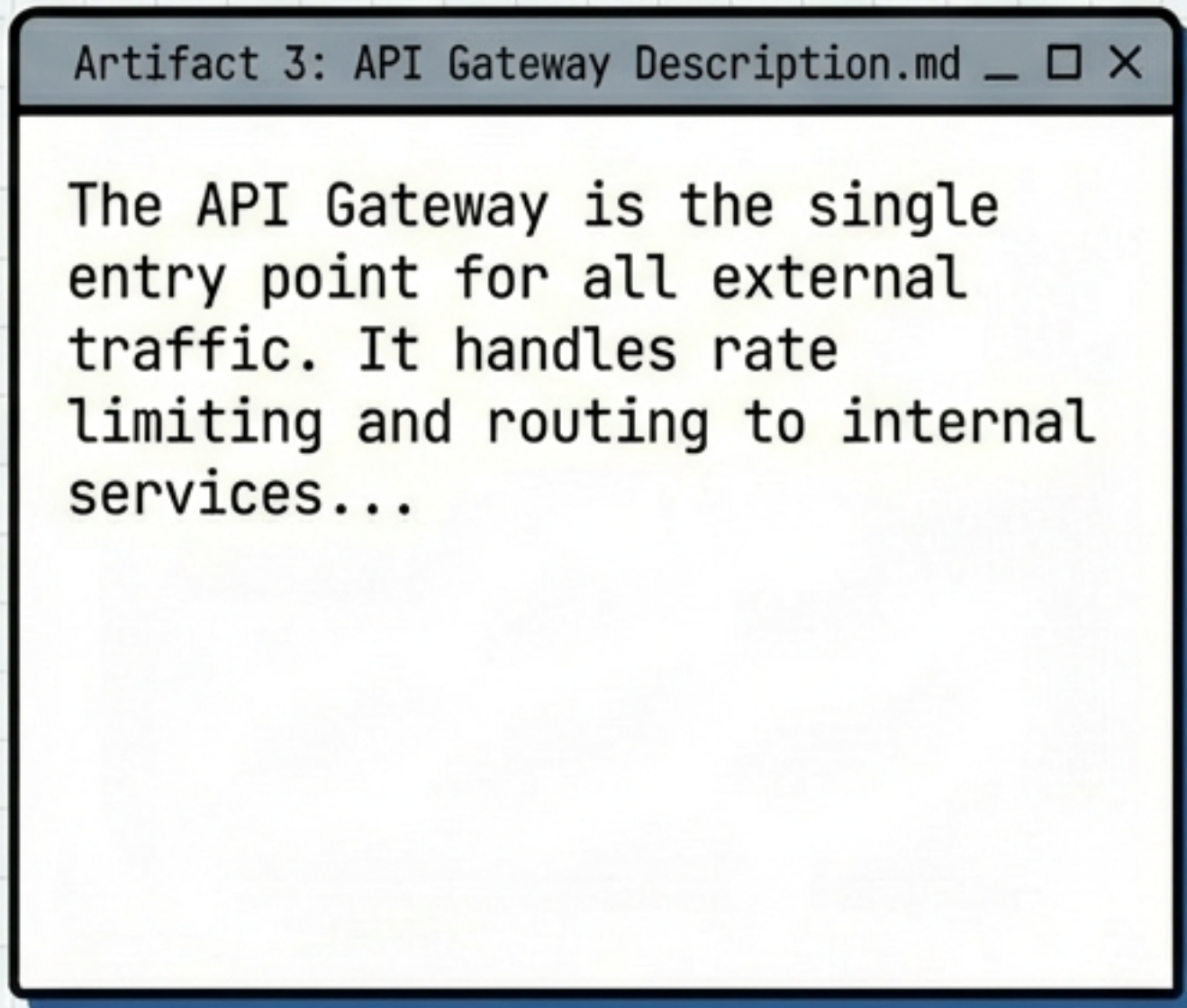
The Output

```
{
  "nodes": ["Customer",
    "Food Platform"],
  "edges": [
    {"from": "Customer",
      "to": "Platform",
      "label": "places order"}]
}
```

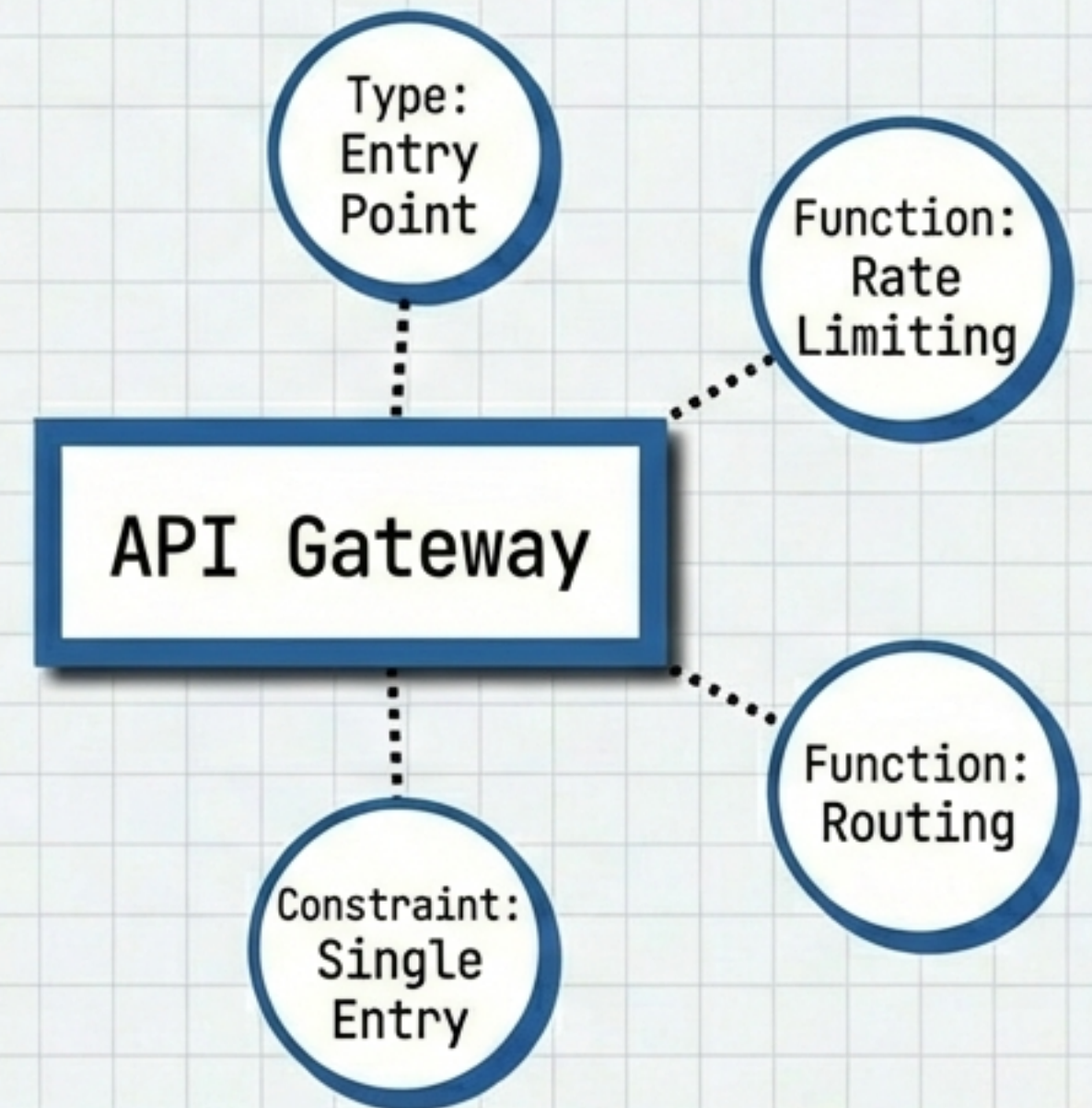


Step 2: Transforming Descriptions into Structure

Artifact 3: API Gateway Description

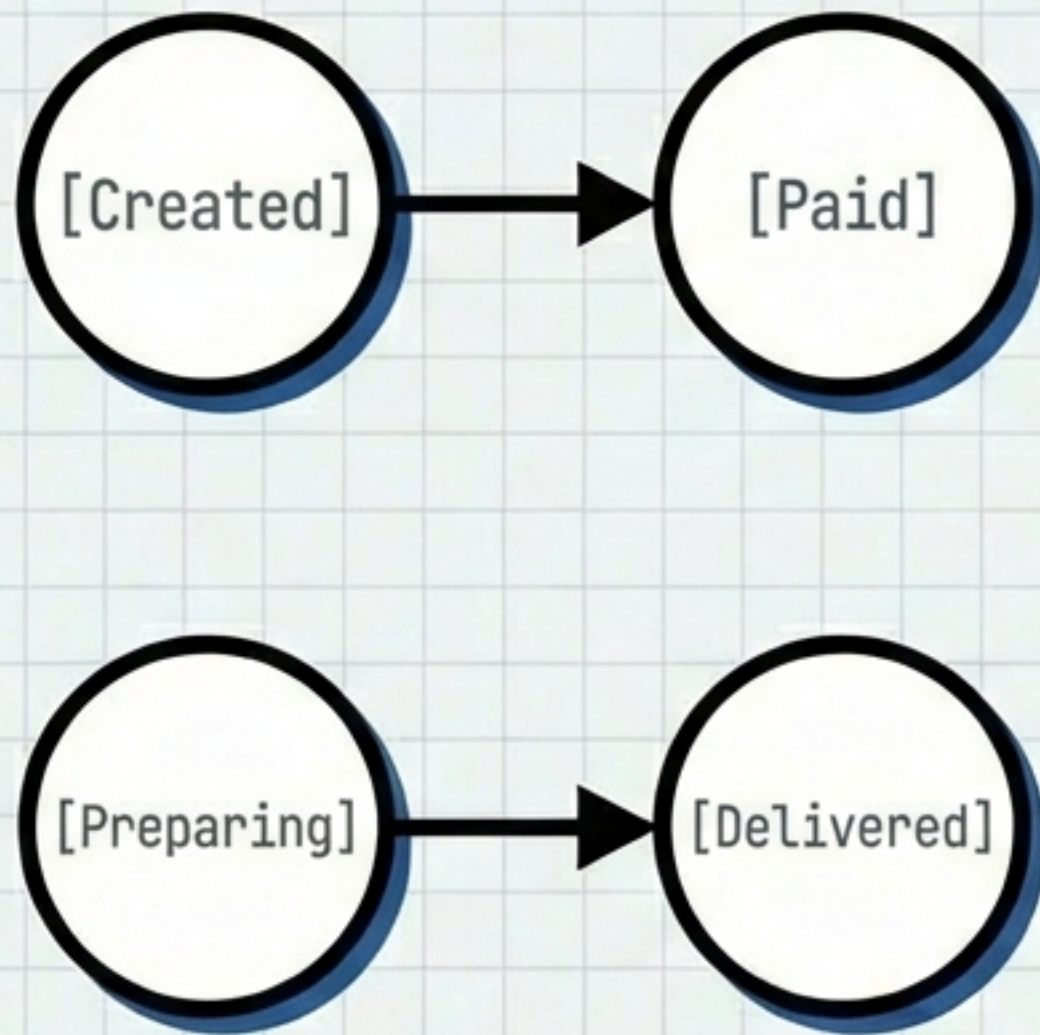


Resulting Graph

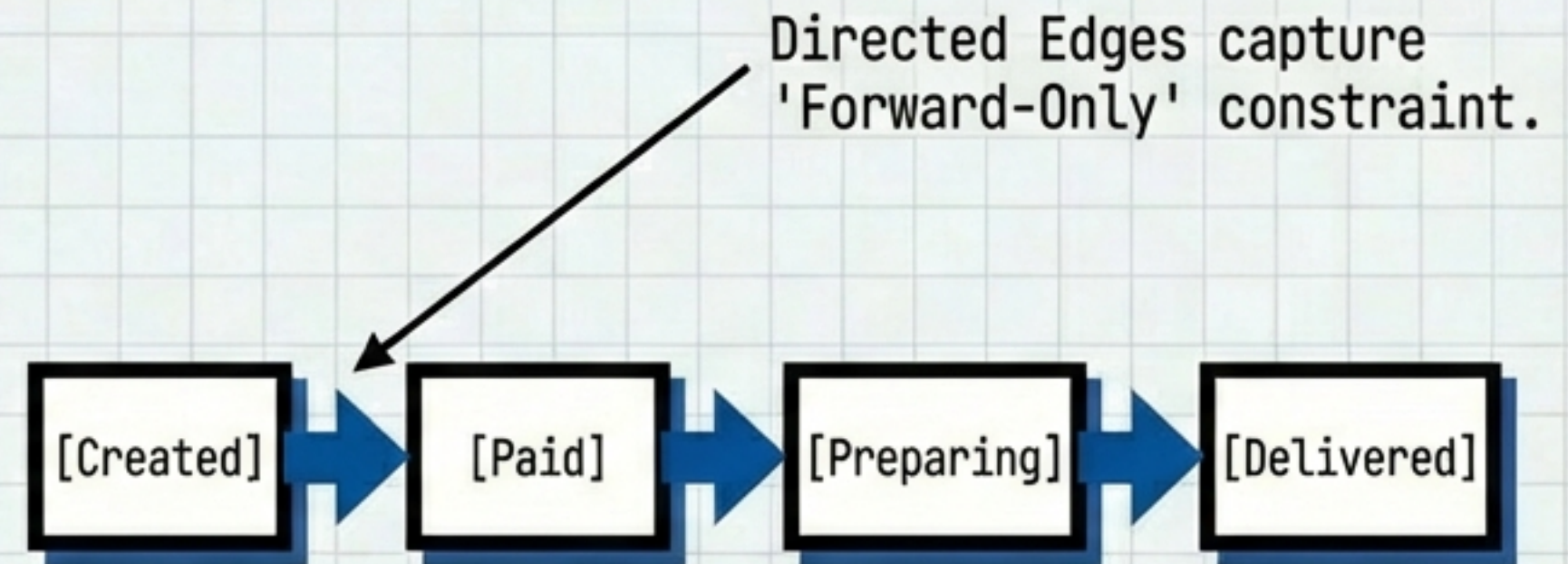


Step 3: Capturing Behaviour and Flow

Artifact 5: Order State Diagram



Resulting Graph

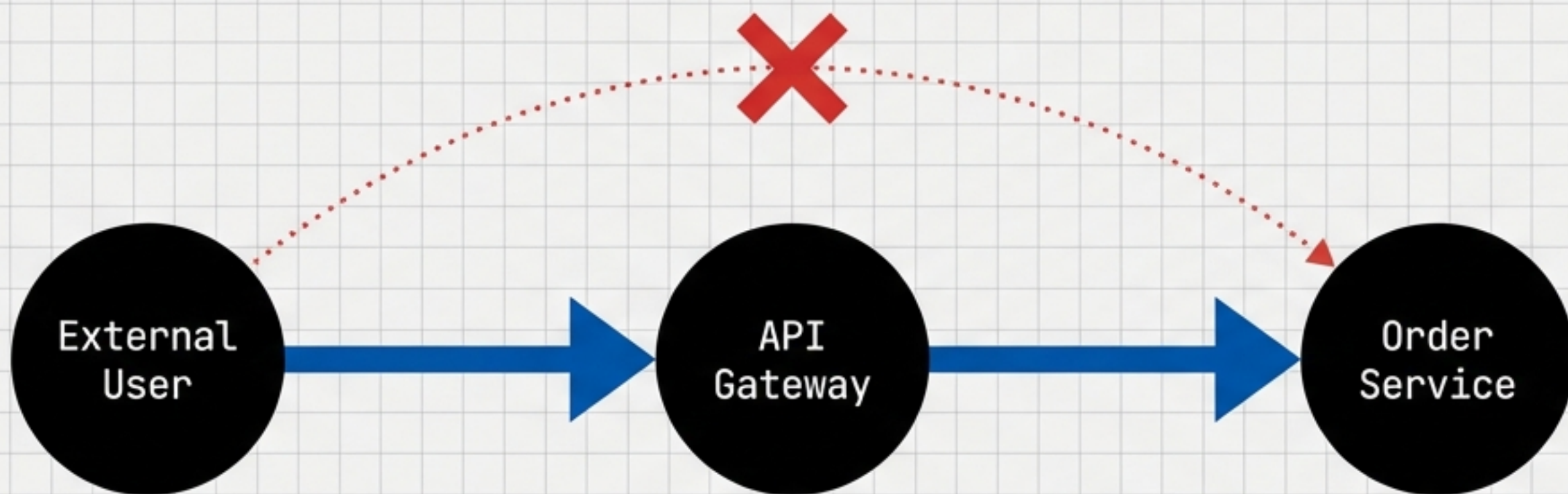


Step 4: Making “Must” and “Never” Testable

Human Rule (Artifact 7)	Graph Query (Executable Test)
All external traffic must pass through API Gateway.	<pre>MATCH path = (e:External)-[r]->(i:Internal) WHERE NOT 'API Gateway' IN nodes(path) RETURN path</pre>
Card numbers never enter our system.	<pre>MATCH (d:Data {type: 'Card Number'})- [r]->(s:System {loc: 'Internal'}) RETURN d, s</pre>

Compatibility Check 1: Structural Integrity

Rule: All external traffic must pass through API Gateway.



[PASSED]

Query Result:

No direct edges found between External and Internal excluding Gateway.

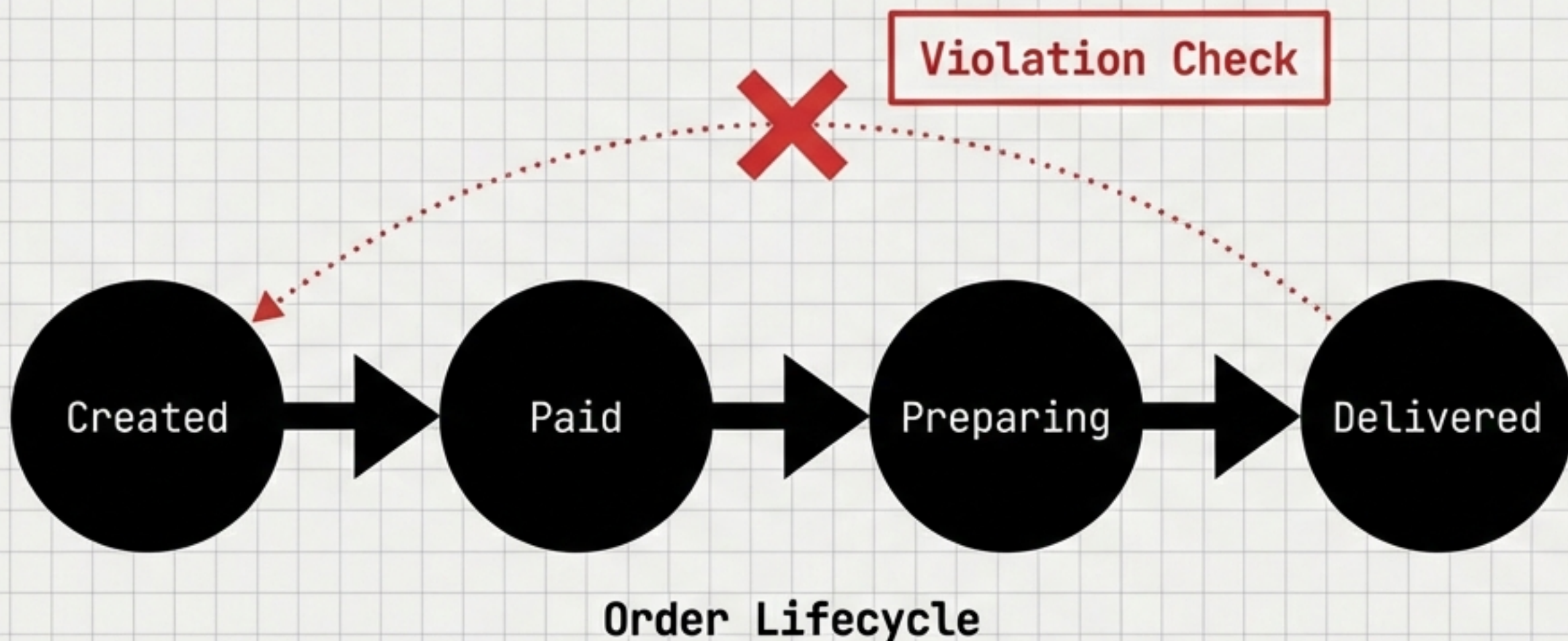
Agent:

CompatibilityEngine

Why This Matters: This is the first step in reverse-engineering the system model for validation.

Compatibility Check 2: Logic & State

Rule: Orders can only move forward.



[PASSED]

Query Result:

Graph is acyclic.
No reverse edges
detected.

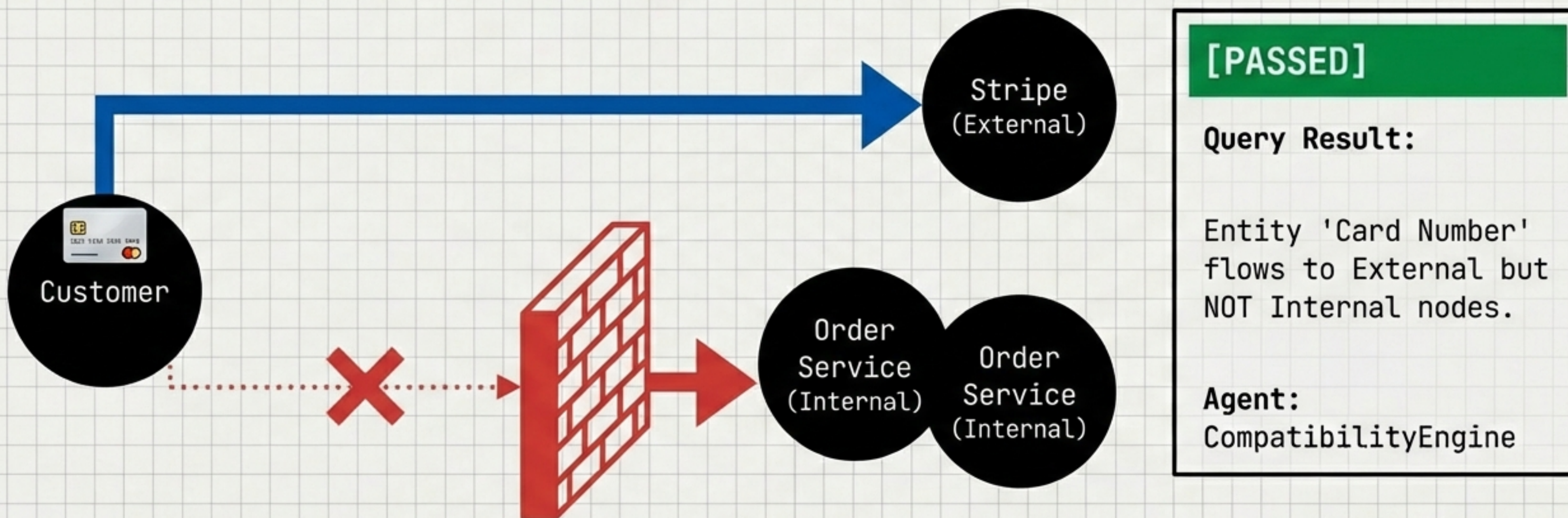
Agent:

CompatibilityEngine

Why This Matters: Ensures the state machine adheres to defined, logical progressions.

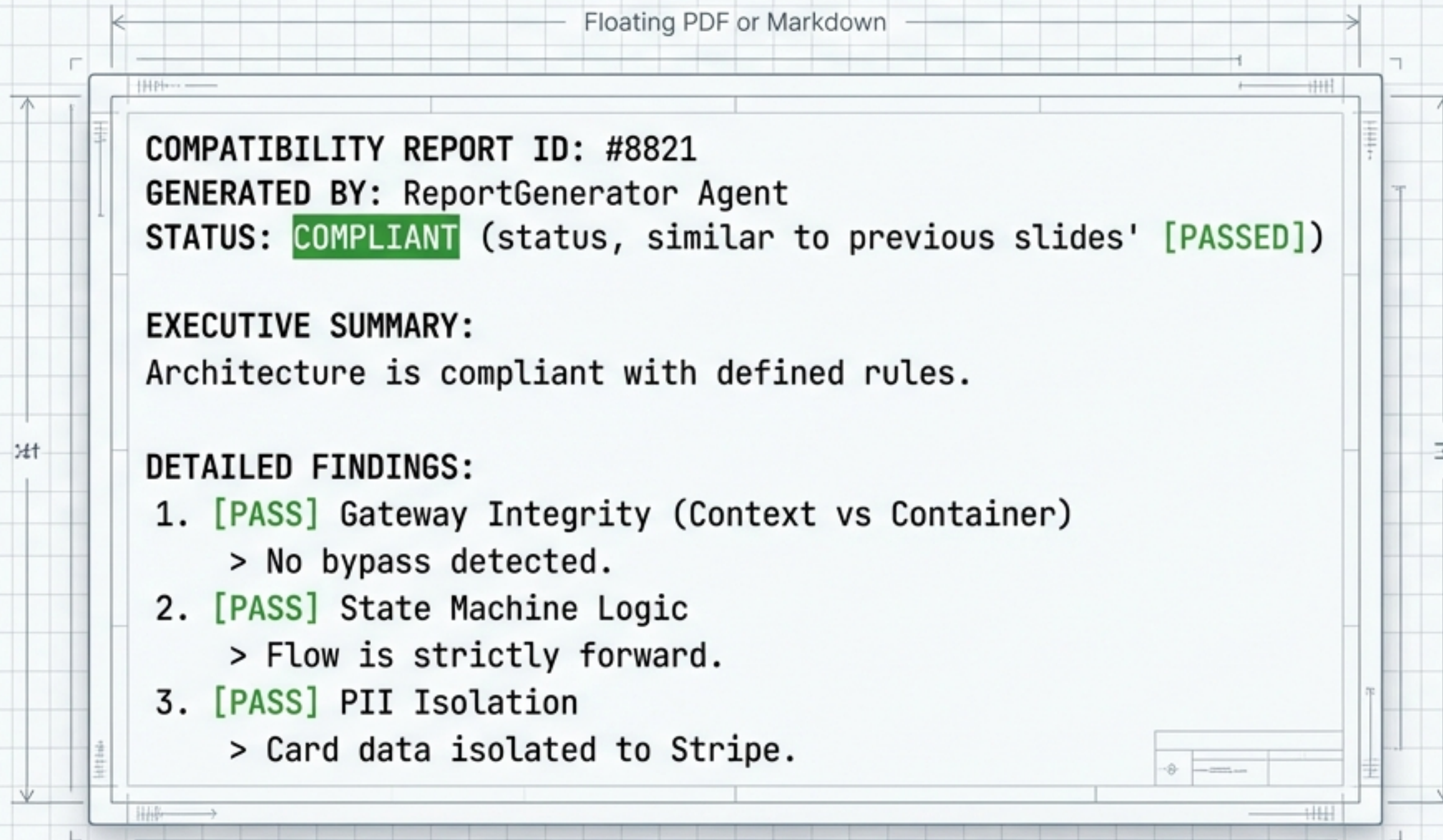
Compatibility Check 3: Data Security

Rule: Card numbers never enter our system.



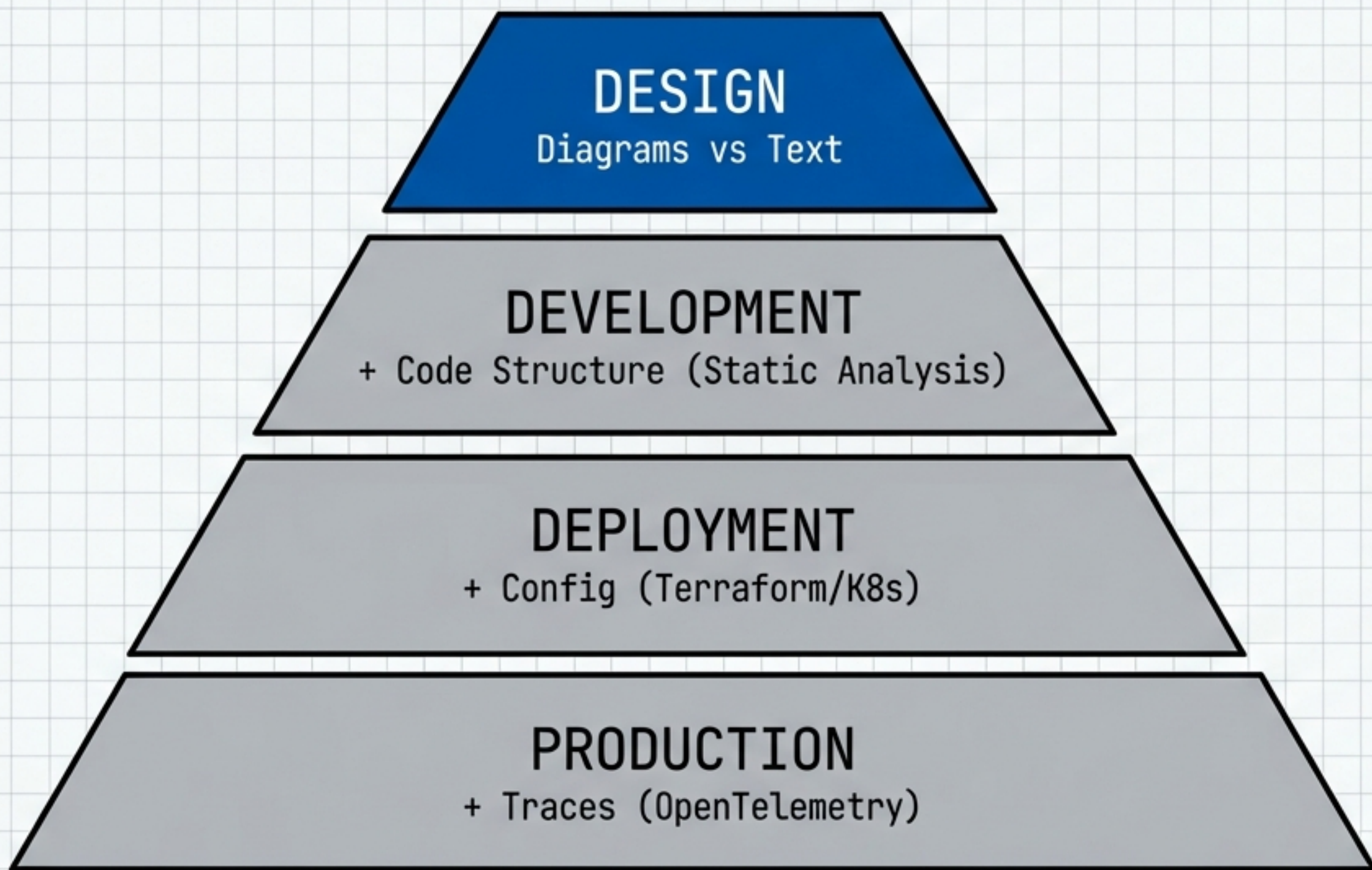
Why This Matters: Protects sensitive data and ensures PCI compliance.

The Final Output: The Compatibility Report



This report was generated by the agent analyzing the graph comparison results. No custom code required.

From Hello World to Production



Full compatibility across layers means: what was **designed is what was built, what was deployed, and what is running.**

References: Compatibility Through Connectivity | LLM as Execution Engine | Thinking in Graphs